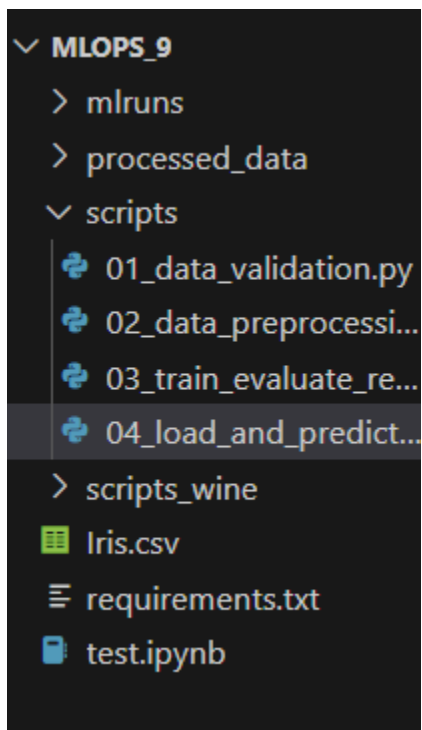


Lab 09: MLFLOW

1. ค้นหา public dataset ที่เป็นตารางข้อมูล สำหรับ ทำ classification และ ใช้ MLFLOW เพื่อสร้างโมเดล และ API สำหรับ การทำ data classification
 - 1.1 แหล่งที่มาของ Dataset (link)
[Iris Species](#)
 - 1.2 Captures โครงสร้างโฟลเดอร์ (แสดงชื่อไฟล์)



1.3 วางโค้ดทั้ง 4 ไฟล์

Validation

```
import pandas as pd
import mlflow

def validate_data():
    """
    Loads the iris dataset, performs basic validation checks,
    and logs the results to MLflow.
    """
    # Set the experiment name for this step
    mlflow.set_experiment("Iris - Data Validation")

    with mlflow.start_run():
        print("Starting data validation run...")
        mlflow.set_tag("ml.step", "data_validation")

        # 1. Load data as a Pandas DataFrame
        df = pd.read_csv("D:/KKU/ปี3 เทอม1/MLOps/lab/MLOPS_9/Iris.csv")
        print("Data loaded successfully.")

        # 2. Perform simple validation checks
        num_rows, num_cols = df.shape
        num_classes = df['Species'].nunique()
        missing_values = df.isnull().sum().sum()

        print(f"Dataset shape: {num_rows} rows, {num_cols} columns")
        print(f"Number of classes: {num_classes}")
        print(f"Missing values: {missing_values}")

        # 3. Log validation results to MLflow
        mlflow.log_metric("num_rows", num_rows)
        mlflow.log_metric("num_cols", num_cols)
        mlflow.log_metric("missing_values", missing_values)
        mlflow.log_param("num_classes", num_classes)

        # Check if the data passes our defined criteria
        validation_status = "Success"
        if missing_values > 0 or num_classes < 3:
            validation_status = "Failed"

        mlflow.log_param("validation_status", validation_status)
        print(f"Validation status: {validation_status}")
        print("Data validation run finished.")

if __name__ == "__main__":
    validate_data()
```

Preprocessing

```
import os
import pandas as pd
from sklearn.model_selection import train_test_split
import mlflow

def preprocess_data(test_size=0.25, random_state=42):
    """
    Loads raw data, splits it into training and testing sets,
    and logs the resulting datasets as artifacts in MLflow.
    """

    # Set the experiment name
    mlflow.set_experiment("Iris - Data Preprocessing")

    with mlflow.start_run() as run:
        run_id = run.info.run_id
        print(f"Starting data preprocessing run with run_id: {run_id}")
        mlflow.set_tag("ml.step", "data_preprocessing")

        # 1. Load data as a DataFrame
        df = pd.read_csv("D:/KKU/13 WAP1/MLOps/lab/MLOPS_9/Iris.csv")

        # 2. Split the data into training and testing sets
        X = df.drop('Species', axis=1)
        y = df['Species']
        X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=test_size, random_state=random_state, stratify=y)

        # 3. Create a temporary directory to save processed data
        processed_data_dir = "processed_data"
        os.makedirs(processed_data_dir, exist_ok=True)

        # Recombine features and target for easy saving
        pd.concat([X_train, y_train], axis=1).to_csv(os.path.join(processed_data_dir, "train.csv"), index=False)
        pd.concat([X_test, y_test], axis=1).to_csv(os.path.join(processed_data_dir, "test.csv"), index=False)
        print(f"Saved processed data to '{processed_data_dir}' directory.")

        # 4. Log parameters and metrics
        mlflow.log_param("test_size", test_size)
        mlflow.log_metric("training_set_rows", len(X_train))
        mlflow.log_metric("test_set_rows", len(X_test))

        # 5. Log the processed data directory as an artifact
        mlflow.log_artifacts(processed_data_dir, artifact_path="processed_data")
        print("Logged processed data as artifacts in MLflow.")

        print("-" * 50)
        print(f>Data preprocessing run finished. Please use the following Run ID for the next step:")
        print(f"Preprocessing Run ID: {run_id}")
        print("-" * 50)

if __name__ == "__main__":
    preprocess_data()
```

Train_and_Evaluate

```
# 2. สร้าง Scikit-learn Pipeline
pipeline = Pipeline([
    ('scaler', StandardScaler()),
    ('model', LogisticRegression(C=C, random_state=42, max_iter=10000))
])
pipeline.fit(X_train, y_train)

# 3. ประเมินผลโมเดล
y_pred = pipeline.predict(X_test)
acc = accuracy_score(y_test, y_pred)
print(f"Accuracy: {acc:.4f}")

# 4. Log Parameters, Metrics, และ Model (Pipeline)
mlflow.log_param("C", C)
mlflow.log_metric("accuracy", acc)
mlflow.sklearn.log_model(pipeline, "iris_classifier_pipeline")

# 5. ตรวจสอบและลงทะเบียนโมเดล
if acc >= ACCURACY_THRESHOLD:
    print(f"Model accuracy ({acc:.4f}) meets the threshold. Registering model...")
    model_uri = f"runs://{mlflow.active_run().info.run_id}/iris_classifier_pipeline"
    registered_model = mlflow.register_model(model_uri, "iris-classifier-prod")
    print(f"Model registered as '{registered_model.name}' version {registered_model.version}")
else:
    print(f"Model accuracy ({acc:.4f}) is below the threshold. Not registering.")
print("Training run finished.")

if __name__ == "__main__":
    if len(sys.argv) < 2:
        print("Usage: python scripts/03_train_evaluate_register.py <preprocessing_run_id> [C_value]")
        sys.exit(1)

    run_id = sys.argv[1]
    c_value = float(sys.argv[2]) if len(sys.argv) > 2 else 1.0
    train_evaluate_register(preprocessing_run_id=run_id, C=c_value)
```

Load_and_predict

```
import mlflow
import pandas as pd

def load_and_predict():
    """
    Simulates a production scenario by loading a model from a specific
    stage in the MLflow Model Registry and using it for prediction.
    """
    MODEL_NAME = "iris-classifier-prod"
    MODEL_STAGE = "Staging" # Change to "Production" after transitioning the model stage

    print(f>Loading model '{MODEL_NAME}' from stage '{MODEL_STAGE}'...")

    # Load the model from the Model Registry
    try:
        model = mlflow.pyfunc.load_model(model_uri=f"models://{MODEL_NAME}/{MODEL_STAGE}")
    except mlflow.exceptions.MlflowException as e:
        print(f"\nError loading model: {e}")
        print(f>Please make sure a model version is in the '{MODEL_STAGE}' stage in the MLflow UI.")
        return

    # Prepare new sample data for prediction (using the first row of the dataset)
    df = pd.read_csv("D:/KKU/ปี3 เทอม1/MLOps/lab/MLOPS_9/Iris.csv")
    X, y = df.drop('Species', axis=1), df['Species']
    sample_data = X[0:1] # Using the first row as a sample
    actual_label = y[0]

    # Use the loaded model to make a prediction
    # No manual preprocessing is needed because we logged the entire pipeline
    prediction = model.predict(sample_data)

    print("-" * 30)
    print(f>Sample Data Features:\n{sample_data[0]}")
    print(f>Actual Label: {actual_label}")
    print(f>Predicted Label: {prediction[0]}")
    print("-" * 30)

if __name__ == "__main__":
    load_and_predict()
```

1.4 Capture ผลลัพธ์การทำงานจากคำสั่ง python scripts/04_load_and_predict.py)

```
(mlflow) D:\KKU\ใบ้ เอนม1\MLOps\lab\MLOPS_9>python scripts/04_load_and_predict.py
Loading model 'iris-classifier-prod' from stage 'Staging'...
C:\Users\gotxz\anaconda3\envs\mlflow\lib\site-packages\sklearn\utils\validation.py:2749:
Scaler was fitted with feature names
  warnings.warn(
-----
Sample Data Features:
[1.  5.1 3.5 1.4 0.2]
Actual Label: Iris-setosa
Predicted Label: Iris-setosa
-----
```

1.5 Captures โมเดลใน MLFLOW ทุกเวอร์ชัน

[Registered Models](#) >

iris-classifier-prod

Created Time: 09/15/2025, 10:49:49 AM

Last Modified: 09/15/2025, 10:51:24 AM

> Description [Edit](#)

> Tags

▼ Versions All Active 1 Compare

Version	Registered at 	Created by	Stage	Description
 Version 2	09/15/2025, 10:51:11 AM		Staging	
 Version 1	09/15/2025, 10:49:49 AM		None	

< Previous Next >

25 / page ▼